

Ereditarietà delle classi

Introduzione delle Ereditarietà delle classi in Python

Introduzione

Abbiamo già affrontato il concetto di ereditarietà delle classi nella lezione [Python/Lezione 6_ Le Classi_ Gli attributi pubblici, privati, gli attributi di classe e i metodi di classe/Le Classi](#) in Python e nella lezione [Generalizzazioni](#) in Progettazione.

In questa lezione andremo a vedere come implementare l'ereditarietà della classi in Python.

Cos'è l'ereditarietà?

È un meccanismo che permette a una classe figlia di riutilizzare metodi e attributi definiti in una classe padre, riducendo la ridondanza e favorendo la riusabilità del codice.

Sintassi



Python

```
1 class Baseclass:
2     def method_base(self)->None:
3         pass
4 class Derivedclass(Baseclass):
5     def method_defived(self)->None:
6         pass
```

Componenti	Descrizione
BaseClass	Il padre(superclasse) che definisce il comportamento condiviso
DerivedClass	Il figlio (sottoclasse) che eredita dalla BaseClass
:	Dichiara l'inizio della definizione della classe
()	Usata per specificare la classe padre che viene ereditata

Esempio di ereditarietà



Python

```
1 class Animal:
2     def speak(self) -> None:
3         print("The animal makes a sound")
4
5 class Dog(Animal):
6     def bark(self) -> None:
7         print("Woof!")
8
9
10 fido: Dog = Dog()
11 fido.speak() # Output: "The animal makes a sound"
12 fido.bark() # Output: "Woof!"
```

In questo esempio, `fido` è un oggetto della classe `Dog`. Possiamo notare che:

- `fido` può **chiamare il metodo** `speak()` definito nella classe `Animal`, perché `Dog` **eredita** da `Animal`;
- `fido` può anche **chiamare** `bark()`, definito direttamente nella classe `Dog`.

Questo accade perché nella definizione della classe `Dog`, abbiamo scritto `class Dog(Animal)`, cioè stiamo dicendo che `Dog` è una **sottoclasse di** `Animal`. In altre parole, `Dog` **is-a** `Animal` (relazione di generalizzazione → **is-a**).

Il concetto di ereditarietà delle classi è rappresentato tramite il **sillogismo aristotelico**:

Il sillogismo aristotelico è una forma di ragionamento deduttivo composta da due premesse (una maggiore e una minore) e una conclusione che deriva logicamente da esse.

È uno strumento classico della logica formale, introdotto da **Aristotele**, per dedurre una verità a partire da affermazioni generali.

Struttura del sillogismo aristotelico

1. **Premessa maggiore**: affermazione generale.
2. **Premessa minore**: affermazione specifica.
3. **Conclusione**: deduzione logica.

L'esempio canonico di questo sillogismo è:

1. **Premessa maggiore**: Tutti gli uomini sono mortali
2. **Premessa minore**: Socrate è un uomo.

3. **Conclusione:** Quindi Socrate è un uomo

Nel nostro caso possiamo usare il sillogismo aristotelico per dedurre che:

1. **Tutti gli animali hanno il metodo** `speak()`
2. **Dog eredita da Animal**
3. **Quindi Dog eredita anche il metodo** `speak()` .

✍ **Questo sillogismo può risultare debole, perché difatti lo è: >**

Deriva da una capacità funzionale da una struttura, non è formalmente deduttivo.

In altre parole Dog eredita anche il metodo `speak()` è una conseguenza comportamentale, vera nel codice, ma meno rigorosa in logica pura.

Questo perché:

- Non tratta **categorie ontologiche** (come "uomo", "mortale", "filosofo" ecc.).
- Ma descrive un **meccanismo tecnico di ereditarietà** (una regola del linguaggio di programmazione).

Quindi:

- ✓ **È corretto nel contesto della OOP** (è "logico" all'interno delle regole della programmazione).
- ✗ **non è logica deduttiva pura aristotelica**, perché non si basa su inclusioni di insiemi concettuali stabili, bensì su una "propagazione di comportamenti" data da un sistema formale (il linguaggio).

Infatti questo tipo di ereditarietà non è una relazione `is-a` ma è

Ereditarietà comportamentale (`has-a method`):

relazione pratica e funzionale in cui una classe eredita

metodi/attributi da un'altra per riusare il comportamento.

Questo tipo di relazione è fondamentale nella OOP, ma differisce dalla relazione `is-a` in quanto non implica necessariamente una categorizzazione concettuale.

☰ **Esempio:**

Dog eredita `speak()` da `Animal`, ma non significa che la logica `is-a` sia sempre corretta.

Concetto	is-a (Relazione logica/strutturale)	Ereditarietà comportamentale (has-a method)
Definizione	Relazione concettuale tra due entità	Condivisione di metodi/attributi tramite ereditarietà
Origine	Logica formale, filosofia	Programmazione ad oggetti (OOP)
Scopo	Categorizzare, creare gerarchie logiche	Riutilizzare codice, condividere funzionalità
Implementazione in OOP	Tramite <code>class</code> Sottoclasse(Superclasse)	Tramite ereditarietà (stessa sintassi)
✓ Quando è corretta	Quando la sottoclasse è un tipo della superclasse	Quando la sottoclasse ha bisogno dei metodi della superclasse
⚠ Rischi	Pochi, se usata con coerenza concettuale	Se usata solo per "riciclare codice", porta a design debole
Esempio valido	Dog is-a Animal	Dog eredita <code>speak()</code> da Animal
⊘ Esempio scorretto	Button is-a Clickable (non sempre vero logicamente)	Button eredita <code>onClick()</code> solo per riuso

🔑 Regola Chiave del sillogismo aristotelico

La **premessa minore** (in questo caso "il cane è un animale") deve sempre incastrarsi con la **premessa maggiore** (in questo caso "tutti gli animali fanno X"), per dedurre la **conclusione logica corretta** ("che il cane fa X")

🔗 La relazione "is-a" (è un) nel sillogismo aristotelico

Non esiste una regola specifica per dove può trovarsi la relazione ``is-a`` all'interno del sillogismo:

può trovarsi sia nella premessa minore che nella conclusione, dipende dal tipo di sillogismo che si va a formulare.

Nel esempio canonico:

1. **Tutti gli uomini sono mortali** (relazione "uomo $\rightarrow$ mortale").
2. **Socrate è un uomo**(relazione `is-a`).
3. **Quindi Socrate è mortale**

Qui la relazione `is-a` (Socrate è un uomo) è nella premessa minore, la conclusione, invece, è in inferenza di proprietà ereditate ("Socrate è mortale").

Anche nel esempio tra la classe `Dog` e la classe `Animal` la relazione `is-a` si trova comunque nella **premessa minore**, ma se si formulasse un diverso sillogismo tra queste due classi, ovvero:

- Premessa maggiore: Tutti i cani sono animali
- Premessa minore: Fido è cane
- Conclusione: Quindi Fido è un animale $\rightarrow$ `is-a`

In questo caso è la conclusione ad esprimere la relazione `is-a` .

☰ In sintesi

Dove si trova	È corretto	Quando succede?
Premessa minore	✓	Quando si descrive un caso particolare ("X è un Y")
Conclusione	✓	Quando deduci una gerarchia o appartenenza ("X è anche un Z")

Il metodo `super().__init__`

L'ereditarietà in Python non si limita ai **metodi**, ma si estende anche agli **attributi** definiti nel costruttore `__init__` della superclasse.

Per inizializzare correttamente la superclasse all'interno di una sottoclasse, si

utilizza il metodo `super().__init__()` :

questo metodo permette di invocare il costruttore della superclasse, così da inizializzare correttamente tutti gli attributi e i comportamenti definiti nella classe padre.

In altre parole, consente alla sottoclasse di **riutilizzare il costruttore della superclasse** (`__init__`) senza dover duplicare codice.

Va notato che **i metodi della superclasse sono ereditati automaticamente**, ma il costruttore deve essere invocato esplicitamente.

Spiegazione di come lavora il `super().__init__()`

`super()` restituisce un **oggetto temporaneo** che rappresenta la superclasse della classe corrente, **quindi chiamando il `super().__init__()` si esegue il costruttore della superclasse permettendo di riutilizzare il codice già scritto senza doverlo copiare nella sottoclasse**.

Questo approccio è fondamentale quando la superclasse contiene logica di inizializzazione che si vuole **mantenere e riutilizzare**.

Esempio senza `super().__init__()` :



Python

```
1  # Without super()
2  class Animal:
3      def __init__(self, name:str) -> None:
4          self.name:str = name
5          print("Animal initialized")
6  class Dog(Animal):
7      def __init__(self, name:str) -> None:
8          self.name:str = name
9          print("Dog initialized")
10 fido:Dog = Dog("Rudy")
11 # Output: Dog initialized
```

In questo caso, la classe `Dog` **non richiama il costruttore** della classe `Animal`, quindi si deve ridefinire esplicitamente gli stessi attributi.

Esempio con il `super().__init__()` :



Python

```
1  # With super()
2  class Animal:
```

```

3     def __init__(self, name:str) -> None:
4         self.name:str = name
5         print("Animal initialized")
6
7     class Dog(Animal):
8         def __init__(self, name:str) -> None:
9             super().__init__(name)
10            print("Dog initialized")
11
12    fido:Dog = Dog("Rudy")
13    print(fido)
14    # Output: Animal initialized
15    # Dog initialized

```

In questo esempio:

- `super().__init__(name)` chiama il costruttore della superclasse `Animal` e inizializza `self.name`.
- Quindi si evita la ripetizione del codice e manteniamo la gerarchia di ereditarietà.

 **Nota:** `super()` è particolarmente utile in presenza di più livelli di ereditarietà e in scenari di ereditarietà multipla.

L'overriding dei metodi

Cosa succede se nella classe padre e nella/e classe/i figlia/e si hanno metodi con lo stesso nome e parametri?

Avviene quello che viene chiamato **overriding** del metodo:

L'overriding di un metodo si verifica quando una sottoclasse ridefinisce un metodo già esistente nella superclasse, mantenendo lo stesso nome e gli stessi parametri. In questo caso, la versione della sottoclasse ha la priorità su quella ereditaria: **il metodo ridefinito sovrascrive quello della superclasse.**

La versione che viene definita nella sottoclasse sostituisce quella eredita dalla classe padre.

Lo scopo di questo comportamento è quello di personalizzare o estendere il comportamento di un metodo ereditato dalla classe padre, e di realizzare il **polimorfismo**: **stessa interfaccia, comportamento diverso**.

Quindi quando si ridefinisce un metodo che esiste già nella superclasse, il metodo della sottoclasse prende il posto di quello originale.

Detto in termini pratici:

quando si richiama quel metodo su una istanza della sottoclasse, verrà eseguita la versione della sottoclasse, non quella della superclasse.

In questo esempio possiamo vedere come agisce l'overriding:



Python

```
1 class Animal:
2     def speak(self) -> None:
3         print("The animal makes a sound")
4
5
6 class Cat(Animal):
7     def speak(self) -> None:
8         print("Meow!")
9
10
11 kitty: Cat = Cat()
12 kitty.speak() # Output: "Meow!"
```

In questo esempio:

1. `Animal` è la **superclasse**, e definisce il metodo `speak()`.
2. `Cat` è la **sottoclasse** che eredita da `Animal`, e ridefinisce anch'essa `speak()` con un comportamento diverso.

Quando creiamo un'istanza di `Cat` (`kitty`) e chiamiamo `kitty.speak()`, l'interprete esegue la **versione ridefinita** del metodo, cioè quella di `Cat`.

Questo succede perché Python usa il **Method Resolution Order(MRO)**: nel risolvere un metodo, parte dalla classe **più specifica** (quella dell'istanza) e **risale** la gerarchia (dal basso verso l'alto) fino a trovare il metodo corrispondente.

In altre parole:

1. Cerca nella classe dell'istanza (`Cat`)
2. Se non trovato, risale la gerarchia (`Animal`)



Ordine di risoluzione dei metodi (MRO)

Per visualizzare l'**ordine con cui Python cerca i metodi** nelle classi (in caso di ereditarietà), puoi usare:



Python

```
1 print(NomeDellaSottoclasse.__mro__)
```

Questo restituirà una **tupla** che rappresenta l'ordine di ricerca: dalla classe specifica, fino ad arrivare a `object`.

Esempio pratico:



Python

```
1 class Animal:
2     def speak(self) -> None:
3         print("The animal makes a sound")
4
5 class Cat(Animal):
6     def speak(self) -> None:
7         print("Meow!")
8
9 kitty: Cat = Cat()
10 kitty.speak() # Output: "Meow!"
11 print(Cat.__mro__)
```

Uso di `super()` e MRO

Il comando `super()` serve per richiamare il metodo della superclasse (classe genitore) senza doverla nominare esplicitamente.

È particolarmente utile quando si vuole estendere il comportamento del metodo ereditato, senza sovrascriverlo completamente.

Esempio:



Python

```
1 class Animal:
2     def speak(self) -> None:
3         print("The animal makes a sound")
4
5 class Dog(Animal):
```

```

6         def speak(self) -> None:
7             super().speak() # richiama speak() da Animal
8             print("Woof!")
9
10    fido: Dog = Dog()
11    fido.speak()

```

Perché funziona così? Il metodo `super()` sfrutta il **Method Resolution Order (MRO)** per determinare da quale classe genitore deve partire la ricerca del metodo da eseguire.

✅ `super() + __mro__` = comportamento prevedibile e ordinato nell'ereditarietà

Quindi:

in presenza di **overriding**, la versione più "vicina" all'istanza vince sempre.

🔔 **Importante** L'**overriding** riguarda solo i **metodi**, non gli **attributi**. Per gli attributi si parla di *shadowing* o *hiding*, e il comportamento può variare.

Gli attributi protetti

In Python è possibile definire **attributi protetti**, cioè attributi che **non dovrebbero essere accessibili dall'esterno della classe**, ma che **possono essere usati dalle sottoclassi**.

Un attributo protetto è accessibile solo all'interno della classe in cui è definito e nelle sue sottoclassi.

Per convenzione, un attributo si considera **protetto** se il suo nome inizia con un singolo underscore (`_`), ad esempio `_attribute` o `_method`.

Infatti questa marcatura è utilizzata solo come convenzione e non è effettivamente una restrizione impostata dal linguaggio.

⚠ In Python questa protezione non è forzata dal linguaggio (come avviene in Java o C++), ma è solo una **convenzione** per indicare che l'attributo o metodo **non dovrebbe essere usato direttamente dall'esterno**.

Esempio:



Python

```
1 class Animal:
2     def __init__(self) -> None:
3         self._type: str = "Mammal" # Attributo protetto
4
5     def _sound(self) -> None:      # Metodo protetto
6         print("Generic animal sound")
7
8 class Dog(Animal):
9     def describe(self) -> None:
10        print(f"I am a {self._type}") # Accesso a attributo
    protetto
11        self._sound()                # Chiamata a metodo
    protetto
12
13 fido: Dog = Dog()
14 fido.describe()
15 # Output:
16 # I am a Mammal
17 # Generic animal sound
```

Punti chiave

- L'underscore singolo `_` segnala agli altri sviluppatori che l'attributo/metodo è destinato a un uso interno alla classe o alle sue sottoclassi.
- Tecnicamente, questi elementi rimangono accessibili dall'esterno, ma è considerato una buona pratica rispettarne la visibilità dichiarata.
- Questa convenzione si applica sia ad attributi che a metodi

Quindi la protezione in Python si basa quindi sulla cooperazione tra sviluppatori piuttosto che su meccanismi rigidi del linguaggio.